1 Simply Typed Lambda Calculus

1.1 Syntax

Taking inspiration from [1] and [2] we define the types of the simply typed lambda calculus as:

$$T \coloneqq o \mid i \mid T \to T$$

That is, o is a type, i is a type and given two types A and B, the expression $A \to B$ is also a type. The type o represents the type of propositions, i represents the type of individuals and $A \to B$ represents the type of functions from A to B.

Some examples of types:

$$o \qquad o \to i \qquad o \to i \to o \qquad (o \to i) \to o$$

We consider $\to$ to be right associative, and thus $o \to i \to o$ is the same as $o \to (i \to o)$.

We define the terms of the simply typed lambda calculus (using the Bruijn levels) as:

$$t \coloneqq n \mid t\,t \mid \lambda_T\, t \mid n^T \mid \alpha \mid \forall_T \mid \exists_T \mid \nabla_T$$

where n is a natural number, T is a type and α is one of the following symbols:

$$\alpha \coloneqq \bot \mid \top \mid \wedge \mid \vee \mid \supset$$

Some examples of terms:

$$\lambda_i\, 0 \qquad (\lambda_{i\to o}\, 2)\, 1 \qquad \lambda_o\, 3\ \vee \qquad \lambda_{o\to o}\, \lambda_i\, 0\, \top \qquad \forall_o\, \bot$$

The interpretation of a term depends on a fixed number m of free variables. All the levels which are less then m represent free variables, while the others represent bound variables. For example, if the free variables are 2 (that is, the levels 0 and 1), the terms $\lambda_i\, 0$ and $(\lambda_{i\to o}\, 2)\, 1$ can be interpreted respectively as $\lambda\, x\colon i, y$ and $(\lambda\, x\colon i \to o, x)\, y$. If the free variables are 0, then $\lambda_i\, 0$ corresponds to $\lambda\, x\colon i, x$, while the term $(\lambda_{i\to o}\, 2)\, 1$ makes no sense, because 1 and 2 should be two distinct bound variables.

1.2 Typing

Given a list of types Γ, a natural number n and a type T, we denote the n-th element of Γ by Γ_n and the list constructed by appending the element T to the end of Γ as Γ, T.

Definition 1. Given a list of types Γ, a term t and a type T, we define the

proposition $\Gamma \vdash t : T$ by the following inference rules:

$$\frac{}{\Gamma \vdash n : T} \quad \text{if } \Gamma_n = T$$

$$\frac{\Gamma \vdash s : A \to B \qquad \Gamma \vdash t : A}{\Gamma \vdash s\,t : B}$$

$$\frac{\Gamma, A \vdash t : B}{\Gamma \vdash \lambda_A\, t : A \to B}$$

$$\frac{}{\Gamma \vdash \bot : o} \qquad \frac{}{\Gamma \vdash \top : o}$$

$$\frac{}{\Gamma \vdash \wedge : o \to o \to o} \qquad \frac{}{\Gamma \vdash \vee : o \to o \to o}$$

$$\frac{}{\Gamma \vdash \supset : o \to o \to o}$$

$$\frac{}{\Gamma \vdash \forall_T : (T \to o) \to o}$$

$$\frac{}{\Gamma \vdash \exists_T : (T \to o) \to o}$$

$$\frac{}{\Gamma \vdash \nabla_T : (T \to o) \to o}$$

If the proposition $\Gamma \vdash t : T$ holds, we say that t *has type* T in the *context* Γ.

Some examples:

$$\frac{}{[i;\, o] \vdash 0 : i}$$

$$\cfrac{\cfrac{\cfrac{}{[i \to o;\, i] \vdash 0 : i \to o} \qquad \cfrac{}{[i \to o;\, i] \vdash 1 : i}}{[i \to o;\, i] \vdash 0\,1 : o}}{\cfrac{i \to o \vdash \lambda_i\, 0\,1 : i \to o}{\vdash \lambda_{i \to o}\, \lambda_i\, 0\,1 : (i \to o) \to i \to o}}$$

We could, for example, define $s \vee t$ as a "primitive" term, with the typing rule:

$$\frac{\Gamma \vdash s : o \qquad \Gamma \vdash t : o}{\Gamma \vdash s \vee t : o}$$

and similarly for all the other constant terms. However, especially in formalization, it is easier to have a small core of basic terms to prove properties of (levels, abstraction and application), and have any additional terms simply as constants.

1.3 Substitution

The substitution of the terms $s_0, \ldots, s_n$ with context m into the term t, indicated as $t\,[s_0, \ldots, s_n]_m$, is defined recursively as follows:

$$\begin{aligned}
i\,[s_0, \ldots, s_n]_m &\coloneqq s_i \quad \text{if } i \in \mathbb{N} \text{ and } i \leq n \\
i\,[s_0, \ldots, s_n]_m &\coloneqq i \quad \text{if } i \in \mathbb{N} \text{ and } i > n \\
(t_1\, t_2)\,[s_0, \ldots, s_n]_m &\coloneqq (t_1\,[s_0, \ldots, s_n]_m)\,(t_2\,[s_0, \ldots, s_n]_m) \\
(\lambda_T\, t)\,[s_0, \ldots, s_n]_m &\coloneqq \lambda_T\,(t\,[s_0^+, \ldots, s_n^+, m]_{m+1}) \\
t\,[s_0, \ldots, s_n]_m &\coloneqq t \quad \text{in all other cases}
\end{aligned}$$

where s_k^+ is the term obtained from s_k by increasing all of its bound levels by one, with respect to a context with m free variables.

If we had defined terms like $s \vee t$ as primitive, we would have had to give explicit rules for them in the substitution definition.

Some examples:

$$
\begin{aligned}
(1\,0)\,[\lambda_i\,0;\,1]_2 &= (1\,[\lambda_i\,0;\,1]_2)\,(0\,[\lambda_i\,0;\,1]_2) = 1\,(\lambda_i\,0) \\
(\lambda_o\,1\,0)\,[\lambda_i\,0;\,1]_2 &= \lambda_o\,((1\,0)\,[\lambda_i\,0;\,1;\,2]_3) = \lambda_o\,1\,(\lambda_i\,0) \\
(\lambda_o\,1\,0)\,[\lambda_i\,0]_2 &= \lambda_o\,((1\,0)\,[\lambda_i\,0;\,2]_3) = \lambda_o\,2\,(\lambda_i\,0) \\
(\lambda_o\,0)\,[\lambda_i\,0\,1]_2 &= \lambda_o\,(0\,[\lambda_i\,0\,1;\,2]_3) = \lambda_o\,\lambda_i\,0\,1 \\
(\lambda_o\,0)\,[\lambda_i\,0\,1]_1 &= \lambda_o\,(0\,[\lambda_i\,0\,2;\,1]_2) = \lambda_o\,\lambda_i\,0\,2 \\
(\lambda_o\,0)\,[\lambda_i\,0\,0]_0 &= \lambda_o\,(0\,[\lambda_i\,1\,1;\,0]_1) = \lambda_o\,\lambda_i\,1\,1
\end{aligned}
$$

In a substitution $t\,[s_0, \ldots, s_n]_m$, the resulting term and all the terms $s_0, \ldots, s_n$ are meant to be interpreted with m free variables, while the term t is meant to be interpreted with $n+1$ free variables.

Theorem 1. *Let $T_1, \ldots, T_m$ and $S_1, \ldots, S_n$ be types and let t and $s_1, \ldots, s_n$ be terms such that $[T_1;\,\ldots;\,T_m] \vdash s_i : S_i$ for every $i = 1, \ldots, n$. If $[S_1;\,\ldots;\,S_n] \vdash t : T$ then $[T_1;\,\ldots;\,T_m] \vdash t\,[s_1;\,\ldots;\,s_n]_m : T$.*

Theorem 2. *Let $T_1, \ldots, T_m$ and $S_1, \ldots, S_n$ be types and let t and $s_1, \ldots, s_n$ be terms such that $[T_1;\,\ldots;\,T_m] \vdash s_i : S_i$ for every $i = 1, \ldots, n$. If $m \leq n$ and $[T_1;\,\ldots;\,T_m] \vdash t\,[s_1;\,\ldots;\,s_n]_m : T$ then $[S_1;\,\ldots;\,S_n] \vdash t : T$.*

For example, $[o] \vdash 0 : o$ and $[i] \vdash \top : o$ and $[i] \vdash 0\,[\top]_1 : o$, that is, $[i] \vdash \top : o$.

Definition 2. We define the substitution of the term s with contexts m and n into the term t as:

$$t\,[s]_m^n \;:=\; t\,[0;\,\ldots;\,m-1;\,s]_n$$

1.4 Reduction

Definition 3. Given a binary relation on terms R_m, we define another relation step_R^m inductively as follows:

$$\frac{R_m(a,b)}{\mathrm{step}_R^m(a,b)}$$

$$\frac{\mathrm{step}_R^m(a,b)}{\mathrm{step}_R^m(a\,t, b\,t)} \qquad \frac{\mathrm{step}_R^m(a,b)}{\mathrm{step}_R^m(s\,a, s\,b)}$$

$$\frac{\mathrm{step}_R^{m+1}(a,b)}{\mathrm{step}_R^m(\lambda_T\,a, \lambda_T\,b)}$$

Definition 4. We define the binary relation conv_β^m between terms inductively as follows:

$$\frac{}{\mathrm{conv}_\beta^m\big((\lambda_T\,t)\,s, t\,[s]_m^m\big)}$$

Definition 5. We define the binary relation conv_η^m between terms inductively as follows:

$$\frac{}{\mathrm{conv}_\eta^m(\lambda_T\, t\, m, t^-)} \quad \text{if } t \text{ does not contain } m$$

We define step_β^m as $\mathrm{step}_{\mathrm{conv}_\beta^m}$ and step_η^m as $\mathrm{step}_{\mathrm{conv}_\eta^m}$.

1.5 Nominal constants

For every type T let $\pi_T \colon \mathbb{N} \to \mathbb{N}$. The application of π to a term is defined recursively as follows

$$\begin{aligned} \pi.n^T &:= (\pi_T(n))^T \\ \pi.(\lambda_T\, t) &:= \lambda_T\, (\pi.t) \\ \pi.(s\, t) &:= (\pi.s)\, (\pi.t) \\ \pi.t &:= t \quad \text{in all other cases} \end{aligned}$$

We say that two terms a, b are equivalent, written $a \approx b$, if there is a family of bijective functions π_T such that a is λ-equivalent to $\pi.b$.

We define the support of a term t, indicated as $\mathrm{supp}(t)$, as the set of nominal constants appearing in t.

2 G Logic

Let Σ be a list of types, Γ a set of terms and B a term. The expression $\Sigma; \Gamma \longrightarrow B$ is called a *sequent*.

We define a calculus with the following inference rules:

$$\frac{\Sigma \vdash B : o \qquad B \approx C}{\Sigma; B \longrightarrow C}\ \mathrm{id} \qquad \frac{\Sigma; \Delta \longrightarrow B \qquad \Sigma; B, \Gamma \longrightarrow C}{\Sigma; \Delta\, \Gamma \longrightarrow C}\ \mathrm{cut}$$

$$\frac{\Sigma; B, B, \Gamma \longrightarrow C}{\Sigma; B, \Gamma \longrightarrow C}\ cL \qquad \frac{\Sigma \vdash B : o \qquad \Sigma; \Gamma \longrightarrow C}{\Sigma; B, \Gamma \longrightarrow C}\ wL$$

$$\frac{\Sigma \vdash C : o}{\Sigma; \bot \longrightarrow C}\ \bot L \qquad \frac{}{\Sigma; \emptyset \longrightarrow \top}\ \top R$$

$$\frac{\Sigma; B, \Gamma \longrightarrow C \qquad \Sigma; D, \Gamma \longrightarrow C}{\Sigma; B \vee D, \Gamma \longrightarrow C}\ \vee L$$

$$\frac{\Sigma \vdash C : o \qquad \Sigma; \Gamma \longrightarrow B}{\Sigma; \Gamma \longrightarrow B \vee C}\ \vee R \qquad \frac{\Sigma \vdash B : o \qquad \Sigma; \Gamma \longrightarrow C}{\Sigma; \Gamma \longrightarrow B \vee C}\ \vee R$$

$$\frac{\Sigma \vdash D : o \qquad \Sigma; B, \Gamma \longrightarrow C}{\Sigma; B \wedge D, \Gamma \longrightarrow C}\ \wedge L \qquad \frac{\Sigma \vdash B : o \qquad \Sigma; D, \Gamma \longrightarrow C}{\Sigma; B \wedge D, \Gamma \longrightarrow C}\ \wedge L$$

$$\frac{\Sigma; \Gamma \longrightarrow B \qquad \Sigma; \Gamma \longrightarrow C}{\Sigma; \Gamma \longrightarrow B \wedge C}\ \wedge R$$

$$\frac{\Sigma; \Gamma \longrightarrow B \qquad \Sigma; D, \Gamma \longrightarrow C}{\Sigma; B \supset D, \Gamma \longrightarrow C}\ \supset L \qquad \frac{\Sigma; B, \Gamma \longrightarrow C}{\Sigma; \Gamma \longrightarrow B \supset C}\ \supset R$$

$$\frac{\Sigma \vdash t : T \qquad \Sigma; B\,[t]_{|\Sigma|}^{|\Sigma|}, \Gamma \longrightarrow C}{\Sigma; \forall_T\, B, \Gamma \longrightarrow C}\ \forall L$$

$$\frac{\begin{array}{c}\{a_1^{T_1}, \ldots, a_n^{T_n}\} = \mathrm{supp}(B) \\ \Sigma, (T_1 \to \ldots \to T_n \to T); \{x^+ \mid x \in \Gamma\} \longrightarrow B\,[|\Sigma|\, a_1^{T_1}\ \ldots\ a_n^{T_n}]_{|\Sigma|}^{|\Sigma|+1}\end{array}}{\Sigma; \Gamma \longrightarrow \forall_T\, B}\ \forall R$$

$$\frac{\begin{array}{c}\{a_1^{T_1}, \ldots, a_n^{T_n}\} = \mathrm{supp}(B) \\ \Sigma, (T_1 \to \ldots \to T_n \to T); B\,[|\Sigma|\, a_1^{T_1}\ \ldots\ a_n^{T_n}]_{|\Sigma|}^{|\Sigma|+1}, \{x^+ \mid x \in \Gamma\} \longrightarrow C^+\end{array}}{\Sigma; \exists_T\, B, \Gamma \longrightarrow C}\ \exists L$$

$$\frac{\Sigma \vdash t : T \qquad \Sigma; \Gamma \longrightarrow B\,[t]_{|\Sigma|}^{|\Sigma|}}{\Sigma; \Gamma \longrightarrow \exists_T\, B}\ \exists R$$

$$\frac{n^T \notin \mathrm{supp}(B) \qquad \Sigma; B\,[n^T]_{|\Sigma|}^{|\Sigma|}, \Gamma \longrightarrow C}{\Sigma; \nabla_T\, B, \Gamma \longrightarrow C}\ \nabla L$$

$$\frac{n^T \notin \mathrm{supp}(B) \qquad \Sigma; \Gamma \longrightarrow B\,[n^T]_{|\Sigma|}^{|\Sigma|}}{\Sigma; \Gamma \longrightarrow \nabla_T\, B}\ \nabla R$$

An example of $\forall R$:

$$\frac{[o;\ i \to i \to o]; 0^{o \to o}\, 0 \longrightarrow 1\, 0^i\, 0^i}{[o]; 0^{o \to o}\, 0 \longrightarrow \forall_{i \to o}\, 1\, 0^i}\ \forall R$$

In G, I should be able to prove:

$$\nabla_x \nabla_y (x \supset y)$$

Which I cannot do without nominal constants.

We want for every term that occurs inside a sequent to have type o. We check that the all the rules enforce this, focusing on the non trivial ones only.

$$\dfrac{\dfrac{\Sigma\,\sigma \vdash t : T \qquad \Sigma\,\sigma \vdash s\,[0;\,1;\,\ldots;\,|\Sigma\,\sigma| - 1;\,t]_{|\Sigma\,\sigma|} : o}{\Sigma\,\sigma, T \vdash s : o}}{\Sigma\,\sigma \vdash \forall_T\, s : o}$$

2.1 Derivation examples

References

[1] Alonzo Church. "A Formulation of the Simple Theory of Types". In: *The Journal of Symbolic Logic* (1940). URL: https://www.jstor.org/stable/2266170.

[2] Benjamin C. Pierce et al. *Software Foundations, Volume 1: Programming Language Foundations*. 2026. URL: https://softwarefoundations.cis.upenn.edu/plf-current.